

Rotated Rectangle IoU Computation

Gnomonic Projection Pedestrian Detection Project

February 6, 2026

1 Introduction

This document describes the algorithm for computing the Intersection over Union (IoU) metric between two rotated bounding boxes in fisheye image coordinates. The implementation uses OpenCV's `rotatedRectangleIntersection` function for accurate polygon intersection computation.

2 Input Representation

Each rotated bounding box $\mathcal{B}$ is represented by:

$$\mathcal{B} = ((c_x, c_y), (w, h), \theta) \quad (1)$$

where:

- (c_x, c_y) – center coordinates in image space
- (w, h) – width and height of the tight-fit rotated rectangle
- θ – rotation angle in degrees

For two bounding boxes $\mathcal{B}_1$ and $\mathcal{B}_2$, we compute their intersection and union to obtain the IoU metric.

3 Algorithm

3.1 Step 1: Compute Rectangle Areas

The area of each rotated rectangle is simply:

$$A_1 = w_1 \times h_1 \quad (2)$$

$$A_2 = w_2 \times h_2 \quad (3)$$

3.2 Step 2: Compute Intersection

Using OpenCV's `rotatedRectangleIntersection`, we determine the intersection type and geometry:

$$\text{intersectionType}, \mathcal{P}_{\text{int}} = \text{rotatedRectangleIntersection}(\mathcal{B}_1, \mathcal{B}_2) \quad (4)$$

where $\mathcal{P}_{\text{int}}$ is the set of vertices defining the intersection polygon (if it exists).

The intersection type can be:

- **NONE**: No intersection exists $\Rightarrow \text{IoU} = 0$
- **FULL**: One rectangle is fully contained within the other
- **PARTIAL**: Rectangles partially overlap

3.3 Step 3: Compute Intersection Area

3.3.1 Case 1: No Intersection

$$A_{\text{int}} = 0 \quad \Rightarrow \quad \text{IoU} = 0 \quad (5)$$

3.3.2 Case 2: Full Containment

When one rectangle is fully inside the other, the intersection area equals the area of the smaller rectangle:

$$A_{\text{int}} = \min(A_1, A_2) \quad (6)$$

The union area is the area of the larger rectangle:

$$A_{\text{union}} = \max(A_1, A_2) \quad (7)$$

Thus:

$$\text{IoU} = \frac{\min(A_1, A_2)}{\max(A_1, A_2)} \quad (8)$$

3.3.3 Case 3: Partial Overlap

For partial overlap, we compute the area of the intersection polygon using the contour area formula:

$$A_{\text{int}} = \text{contourArea}(\mathcal{P}_{\text{int}}) \quad (9)$$

where $\mathcal{P}_{\text{int}} = \{(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})\}$ is the ordered set of polygon vertices.

The contour area is computed using the Shoelace formula:

$$A_{\text{int}} = \frac{1}{2} \left| \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) \right| \quad (10)$$

where indices are taken modulo n .

3.4 Step 4: Compute Union Area

The union area is computed as:

$$A_{\text{union}} = A_1 + A_2 - A_{\text{int}} \quad (11)$$

3.5 Step 5: Compute IoU

Finally, the Intersection over Union metric is:

$$\text{IoU} = \frac{A_{\text{int}}}{A_{\text{union}}} = \frac{A_{\text{int}}}{A_1 + A_2 - A_{\text{int}}} \quad (12)$$

The result is clamped to the range $[0, 1]$ to handle numerical precision issues:

$$\text{IoU} = \max \left(0, \min \left(1, \frac{A_{\text{int}}}{A_{\text{union}}} \right) \right) \quad (13)$$

4 Edge Cases

1. If $A_{\text{union}} \leq 0$ (degenerate rectangles), we return $\text{IoU} = 0$.
2. If the intersection polygon has fewer than 3 vertices, we return $\text{IoU} = 0$.
3. Any computational errors (invalid bbox format, numerical issues) return $\text{IoU} = 0$.

5 Implementation Notes

- All bounding boxes must be in the same coordinate system (fisheye image coordinates).
- Rotation angles are in degrees, following OpenCV convention.
- The implementation uses OpenCV's optimized C++ backend for polygon intersection.
- Numerical stability is ensured through clamping and error handling.